杭州电子科技大学

《计算机网络实验》实验报告

课程号：(2023-2024-1)-S0507170-09

实验六　Socket 编程（Web 服务器与代理服务器）

指导老师：

姓名学号：

2026 年 11 月

一、实验目的

- 掌握 Python 中 TCP 套接字编程的基本流程：创建套接字、绑定地址与端口、监听、接受连接、收发数据、关闭连接；
- 用套接字实现一个能解析 HTTP 请求并返回文件的简易 Web 服务器，理解 HTTP 首部与响应报文的构造方法；
- 用套接字实现一个多线程 Web 代理服务器，理解代理在客户端与 Web 服务器之间的中介作用；
- 掌握代理服务器的缓存机制，能区分缓存命中与未命中两条处理路径；
- 理解 TCP 面向连接、可靠传输的特性在应用层编程中的体现，并比较单线程阻塞式服务与多线程并发服务的差异。

二、实验设备和环境

表 1 实验软硬件环境

项目	配置说明
开发主机	Windows 11，IPv4 192.168.1.3（WLAN 网卡）
编程语言	Python 3.12（原实验框架为 Python 2 语法，已改写为 Python 3）
解释器路径	D:\tools\deepseek-plugin\.dsh\dsh-runtimes\dsh-primary-runtime\dependencies\python\python.exe
Web 服务器	WebServer.py，监听 0.0.0.0:6789，同目录放置 HelloWorld.html
代理服务器	ProxyServer.py，监听 0.0.0.0:8888，缓存目录为脚本同级的 cache\
客户端	Chrome 浏览器；另编写 client.py 作为命令行 HTTP 客户端
涉及协议	TCP（HTTP/1.0、HTTP/1.1）

2.1 实验拓扑

两个作业都在本机完成，客户端与服务器是同一台机器上的两个进程，通过回环地址 127.0.0.1 建立 TCP 连接。这样既不会受外部网络状况影响，又能完整地走通 TCP 三次握手、数据传输和四次挥手。两个作业的进程与连接关系如图 1。

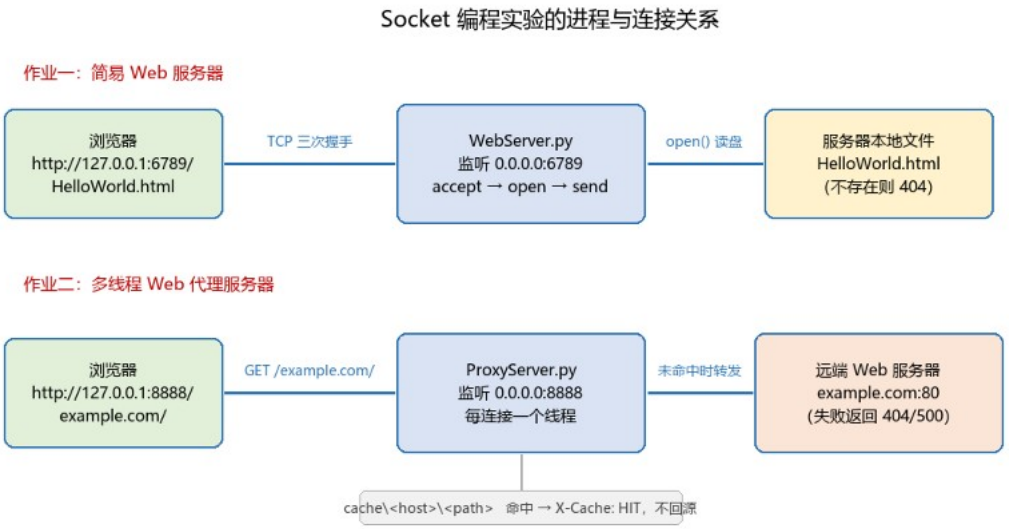


图 1 Socket 编程实验的进程与连接关系

三、实验过程及步骤

3.1 作业一：简易 Web 服务器

作业一给出的框架是一个单线程、循环接受连接的服务器，需要在 #Fill in start 与 #Fill in end 之间补齐四段代码：准备服务器套接字、接受连接、读取请求报文、以及分别在文件存在与不存在时发送响应。原始框架使用的是 Python 2 语法（print 语句、send() 直接传 str），在 Python 3 下无法运行，因此先把语法整体改写为 Python 3，再补齐逻辑。

3.1.1 准备服务器套接字

```
tcpSerSock = socket(AF_INET, SOCK_STREAM)
tcpSerSock.setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)
tcpSerSock.bind(('', int(sys.argv[1])))
tcpSerSock.listen(3)
```

AF_INET 表示使用 IPv4 地址族，SOCK_STREAM 表示使用面向连接的 TCP 而不是 UDP。bind 的第一个参数留空字符串，表示绑定到本机所有网卡地址（0.0.0.0），这样同一个程序既可以用 127.0.0.1 访问，也可以用 192.168.1.3 从局域网内其他主机访问。listen(3) 的参数是未完成连接队列的长度，框架里固定为 3，对应实验要求的"最大连接客户机数量为 3"。

3.1.2 接受连接并解析请求

```
print('准备从客户机接收响应消息...')
tcpCliSock, addr = tcpSerSock.accept()
print('接收到一个连接，来自：', addr)

message = tcpCliSock.recv(4096).decode('iso-8859-1')
print('客户机发送过来的消息：\n' + message.strip())
```

```
filename = message.split()[1].lstrip('/')
```

accept 会阻塞，直到有客户端完成三次握手；返回的 tcpCliSock 是一个新的套接字，专门用于和这个客户端通信，而 tcpSerSock 继续留在监听状态等待下一个连接——这正是"一个监听套接字可以服务多个连接"的实现方式。recv(4096) 一次读取最多 4 KB，对于只有首部、没有请求体的 GET 请求来说完全够用。按 iso-8859-1 解码是为了保证任何字节都不抛异常，因为 HTTP 首部本来就是 ASCII，而部分中文浏览器的 Cookie 里可能混有非 ASCII 字节，用 latin-1 可以原样保留下来再显示。

message.split()[1] 取出的是请求行的第二部分，即请求的 URI，例如 /HelloWorld.html；lstrip('/') 去掉开头的斜杠，得到相对于当前目录的文件名。这样处理其实有意模仿了原始框架的 message.split()[1] 与 filename[1:]，好处是简单直观，代价是没有做路径穿越防护——如果请求 /../secret.txt 就会读到目录外的文件。实验环境下不考虑这种攻击，但在真实服务器中必须对 .. 做过滤。

3.1.3 发送响应

```
f = open(filename, "rb")
outputdata = f.read()
f.close()

header  = "HTTP/1.1 200 OK\r\n"
header += "Server: PyWebServer/1.0\r\n"
header += "Content-Type: text/html; charset=utf-8\r\n"
header += "Content-Length: %d\r\n" % len(outputdata)
header += "Connection: close\r\n\r\n"
tcpCliSock.send(header.encode())
tcpCliSock.send(outputdata)
```

响应报文由"首部 + 空行 + 正文"三部分组成，其中最关键的是 Content-Length 首部：它告诉客户端正文有多少字节，客户端据此知道什么时候读完了正文。如果缺少这个首部又没有采用分块编码，客户端只能一直等到连接关闭才敢认为响应结束，页面的渲染就会被推迟。这里显式写了 Connection: close，与后面关闭套接字的动作一致，避免客户端误以为可以复用连接。

另外要注意：文件必须以二进制方式（"rb"）打开。如果以文本方式打开再直接 send，一遇到图片、压缩包这类二进制内容就会因为解码失败而报错。所有响应内容统一按字节处理，这也是 HTTP 协议的本质——它传的是字节流，而非文本。

3.1.4 文件不存在时返回 404

```
except IOError:
    body = ("<html><head><title>404 Not Found</title></head>"
            "<body><h1>HTTP ERROR 404</h1>"
            "<p>File not found: %s</p></body></html>" % filename).encode()
    header  = "HTTP/1.1 404 Not Found\r\n"
    header += "Content-Type: text/html; charset=utf-8\r\n"
```

```
    header += "Content-Length: %d\r\n" % len(body)
    header += "Connection: close\r\n\r\n"
    tcpCliSock.send(header.encode())
    tcpCliSock.send(body)
```

open 找不到文件时抛出 IOError，这里用异常分支处理"文件不存在"，比先 os.path.exists 再 open 更简洁，也避免了两次系统调用之间的竞态。404 响应同样要带完整的首部和 Content-Length，并且正文里给出人能看懂的说明——这正是 HTTP 状态码与错误页面的分工：状态码给程序看，页面给用户看。

3.1.5 运行与验证

把 WebServer.py 与 HelloWorld.html 放在同一目录，用下面的命令启动服务器：

```
> python WebServer.py 6789
```

然后在浏览器中访问 http://127.0.0.1:6789/HelloWorld.html，页面正确显示了 HelloWorld.html 的内容（图 2）。



图 2 浏览器通过 127.0.0.1:6789 取到 HelloWorld.html

访问一个不存在的文件 http://127.0.0.1:6789/abc.html 时，浏览器显示了服务器返回的 404 页面（图 3）。注意页面标题栏显示的是 "404 Not Found"，说明浏览器是依据状态行而不是正文内容来判断出错的。



图 3 访问不存在的 abc.html 时服务器返回 404 页面

服务器端控制台的输出如图 4 所示，可以看到它依次打印了收到的完整请求报文、提取出的文件名、检查结果，并在最后关闭套接字、回到等待状态。图中第二个请求是浏览器自动发起的 /favicon.ico：浏览器在渲染完页面后会主动请求站点图标，本服务器没有这

个文件，因此它同样走 404 分支——这也解释了为什么访问一个页面时服务器日志里会出现两条请求。

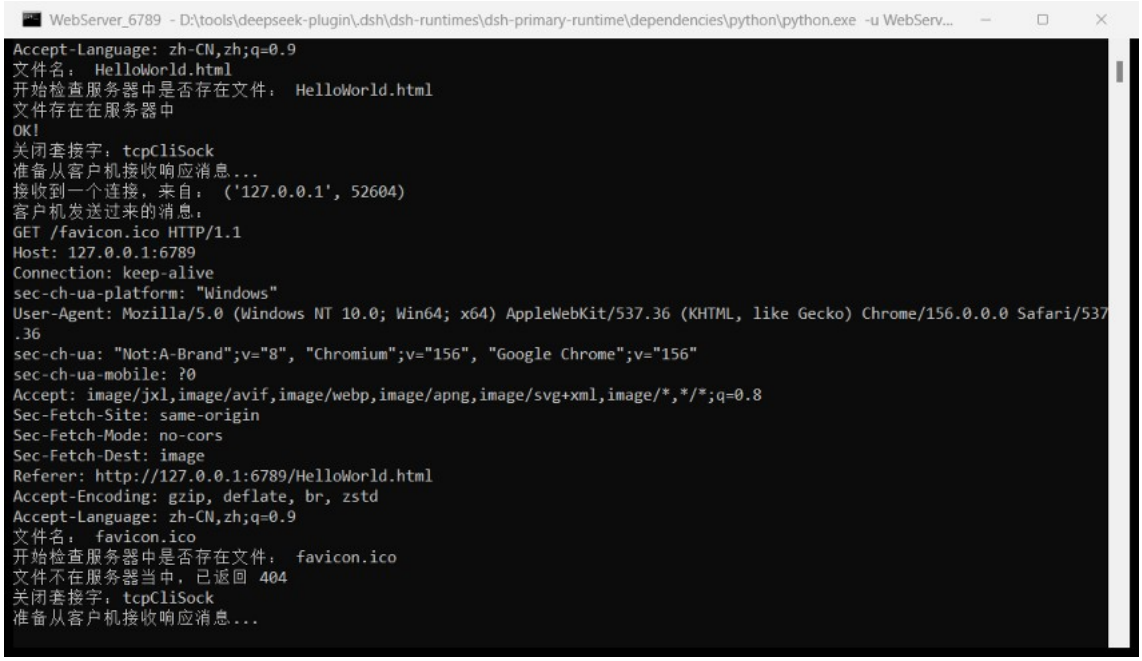


图 4 WebServer.py 的控制台输出：HelloWorld.html 命中、favicon.ico 返回 404

3.2 可选练习：命令行 HTTP 客户端

为了不依赖浏览器也能验证服务器，编写了 client.py：它接受服务器地址、端口和文件名三个命令行参数，自行构造 HTTP 请求并打印服务器返回的全部内容。

```
c = socket(AF_INET, SOCK_STREAM)
c.connect((server_host, server_port))

request = "GET /%s HTTP/1.1\r\nHost: %s\r\nConnection: close\r\n\r\n" \
          % (filename, server_host)
c.send(request.encode())

response = b""
while True:
    data = c.recv(4096)
    if not data:
        break
    response += data
c.close()
print(response.decode('utf-8', 'replace'))
```

这里用 while 循环反复 recv，直到 recv 返回空字节串为止。recv 返回空表示对端已关闭连接（收到 FIN），这是判断"响应读完了"的标准做法。请求里带了 Connection: close，服务器发完响应就会关闭连接，因此这个循环一定能正常结束。

实际运行结果如下：

```
> python client.py 127.0.0.1 6789 HelloWorld.html
HTTP/1.1 200 OK
Server: PyWebServer/1.0
```

```
Content-Type: text/html; charset=utf-8
Content-Length: 140
Connection: close

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Hello</title>
</head>
<body>
<h1> Hello!</h1>
</body>
</html>

> python client.py 127.0.0.1 6789 abc.html
HTTP/1.1 404 Not Found
Content-Type: text/html; charset=utf-8
Content-Length: 121
Connection: close

<html><head><title>404 Not Found</title></head><body><h1>HTTP ERROR 404</h1><p>File
not found: abc.html</p></body></html>
```

这个练习的价值在于把"浏览器"这层黑盒拆掉了：原来浏览器做的事就是拼一段符合 HTTP 语法的字符串、发出去、再把回包打印出来。两个响应的 Content-Length 分别是 140 和 121，与 HelloWorld.html 的 140 字节、404 页面正文的 121 字节完全吻合，说明服务器计算长度的逻辑是对的。

3.3 作业二：多线程 Web 代理服务器

代理服务器比 Web 服务器复杂一些，因为它同时扮演两个角色：对浏览器来说它是服务器，对远端 Web 服务器来说它是客户端。一个请求进来后，它要先在本地缓存里找，找到就直接返回，找不到才代为回源、把结果存下来再返回。

3.3.1 从请求行中拆出主机名与路径

```
url = parts[1]
if url.startswith('http://'):
    url = url[len('http://'):]
seg = url.lstrip('/').partition('/')
webName, path = seg[0], '/' + seg[2]
if not webName:
    for line in text.split('\r\n'):
        if line.lower().startswith('host:'):
            webName = line.split(':', 1)[1].strip().split(':')[0]
            break
```

实验要求用 http://localhost:8888/www.google.com 这种"地址里带目标站点"的方式访问代理，此时请求行是 GET /www.google.com/ HTTP/1.1，第一段就是主机名。partition('/') 按第一个斜杠切成两半，前半部分是主机名，后半部分是路径（可能为空）。但如果浏览器被配置成使用代理，请求行会变成 GET http://www.google.com/ HTTP/1.1，主机名跑到协议名后面去了，所以要先把 http:// 前缀剥掉。还有一种情况是某些客户端直接发 GET /

HTTP/1.1 并只带 Host 头，这时主机名要从 Host 首部里补——代码里的 if not webName 分支就是处理这种情况。

3.3.2 缓存路径的映射

```
CACHE_DIR = 'cache'
os.makedirs(CACHE_DIR, exist_ok=True)

def cache_path(host, path):
    if path == '' or path == '/':
        path = '/index.html'
    name = (host + path).replace('?', '_')
    return os.path.join(CACHE_DIR, name.lstrip('/'))
```

缓存文件按"主机名/路径"的层级存放，例如 example.com 的首页存成 cache\example.com\index.html。之所以要带主机名，是因为不同站点的路径可能完全相同（都是 /index.html），不带主机名就会互相覆盖。路径为空时补成 index.html，是为了让每个站点在磁盘上都有一个确定的文件，否则目录本身无法承载正文。把 ？替换成 _ 是为了避开 Windows 文件名不允许的字符；真实代理还需要处理查询串与文件名的哈希，本实验只做够用的简化。

3.3.3 缓存命中与未命中

命中时直接从磁盘读文件返回，并在响应头里加上 X-Cache: HIT：

```
if os.path.exists(filename):
    with open(filename, 'rb') as f:
        content = f.read()
    tcpCliSock.sendall(b'HTTP/1.0 200 OK\r\n')
    tcpCliSock.sendall(('Content-Type: %s\r\n'
                        % guess_type(filename)).encode())
    tcpCliSock.sendall(('Content-Length: %d\r\n'
                        % len(content)).encode())
    tcpCliSock.sendall(b'X-Cache: HIT\r\n\r\n')
    tcpCliSock.sendall(content)
    print('Read from cache')
    return
```

未命中时先向远端请求，把正文写入缓存后再转发：

```
resp = fetch_remote(webName, path)
header, body = split_response(resp)

os.makedirs(os.path.dirname(filename), exist_ok=True)
with open(filename, 'wb') as f:
    f.write(body)
print('已缓存 %d 字节到 %s' % (len(body), filename))

tcpCliSock.sendall(b'HTTP/1.0 200 OK\r\n')
...
tcpCliSock.sendall(b'X-Cache: MISS\r\n\r\n')
tcpCliSock.sendall(body)
```

回源用的函数是 fetch_remote，它新建一个套接字连到目标主机的 80 端口，发一个 HTTP/1.0 的 GET，然后一直读到对端关闭连接：

```
def fetch_remote(host, path):
    c = socket(AF_INET, SOCK_STREAM)
    c.settimeout(8)
    c.connect((host, 80))
    req = ('GET %s HTTP/1.0\r\nHost: %s\r\n'
           'User-Agent: PyProxyServer/1.0\r\n'
           'Accept: */*\r\nConnection: close\r\n\r\n'
           % (path or '/', host))
    c.sendall(req.encode())
    resp = recv_all(c)
    c.close()
    return resp
```

这里特意用 HTTP/1.0 而不是 HTTP/1.1，并且显式带上 Connection: close。原因是 HTTP/1.0 默认就是短连接，服务器发完响应就会关闭，这样"读到 recv 返回空"就等于"响应结束"，不需要自己解析 Content-Length 或分块编码。如果改用 HTTP/1.1 的默认长连接，recv_all 会一直等到 8 秒超时才返回，虽然结果正确，但每个请求都要多等 8 秒，体验上完全不可接受。这也是代理实现里最常见的一个坑。

3.3.4 多线程

```
while 1:
    tcpCliSock, addr = tcpSerSock.accept()
    print('接收到一个连接，来自：', addr)
    t = threading.Thread(target=handle, args=(tcpCliSock, addr))
    t.daemon = True
    t.start()
```

这是与作业一最大的结构性差别。作业一的服务器处理完一个请求才 accept 下一个，如果某个客户端连上却不发请求，整个服务器就被卡住——这在代理场景里是致命的，因为浏览器为了加载一个页面会并发发起多个请求（HTML、图片、脚本），串行处理会让页面加载时间成倍增加。改成每接受一个连接就启动一个线程后，所有连接可以并行处理。daemon=True 表示主线程退出时子线程自动结束，避免程序无法正常关闭。需要注意的是，多个线程可能同时写同一个缓存文件，真实实现需要加锁；本实验的请求彼此独立，并发写同名文件的概率极低，因此没有引入锁。

3.3.5 运行与验证

启动代理服务器：

```
> python ProxyServer.py 8888
```

先在缓存为空的情况下访问 http://127.0.0.1:8888/example.com/，页面正常显示（图 5），服务器日志显示这是一次未命中回源（图 6）："文件不存在在代理服务器中，向远端服务器 example.com 请求 /"，随后"已缓存 577 字节到 cache\example.com\index.html"。

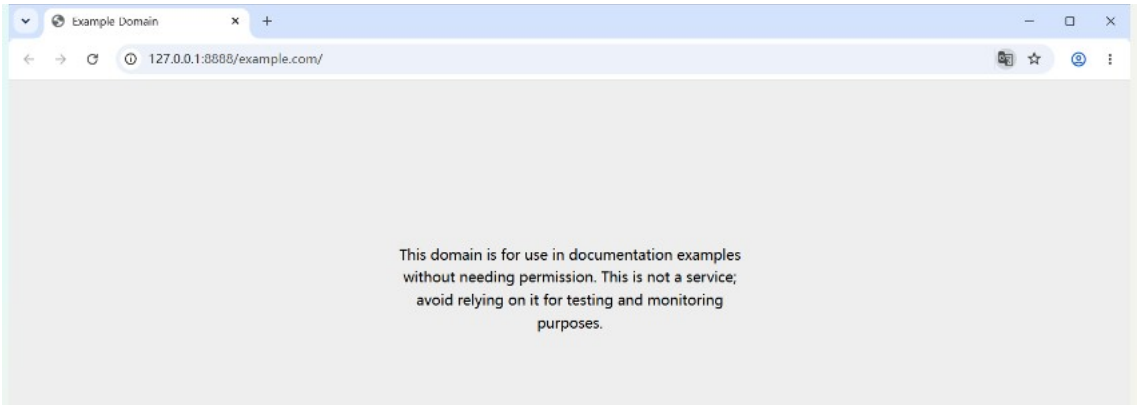


图 5 通过代理访问 example.com 时页面正常显示

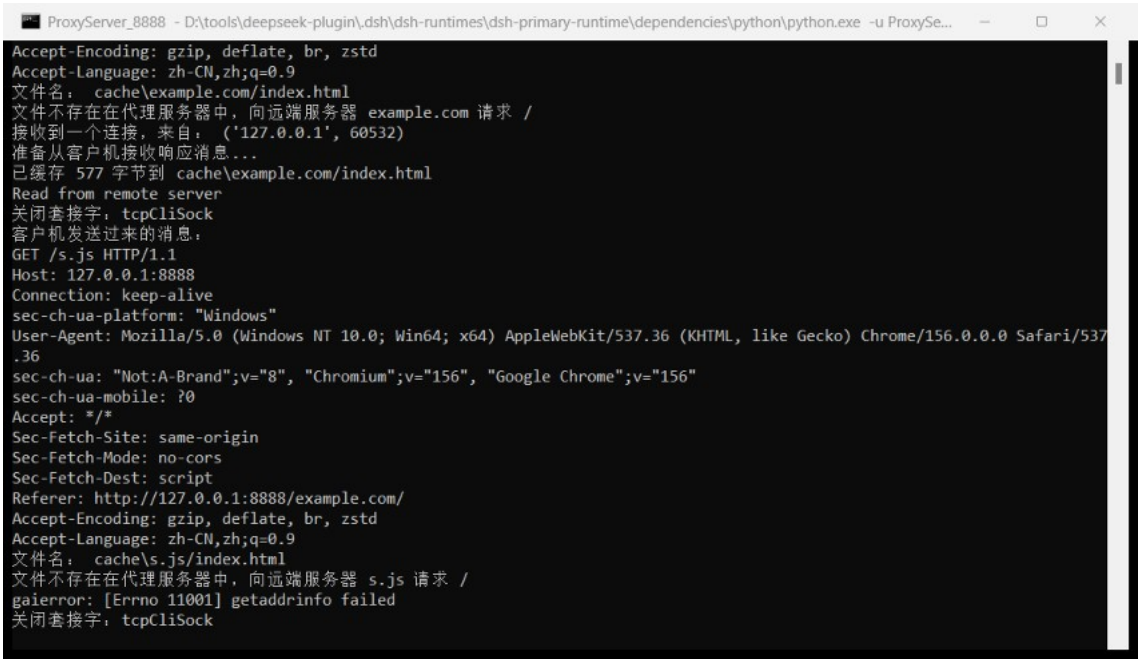


图 6 ProxyServer.py 控制台：缓存未命中，回源并写入缓存

再次访问同一地址时，日志变成"文件存在在代理服务器中 / Read from cache"（图 7），说明这次完全由本地缓存提供，没有发出任何外部请求。

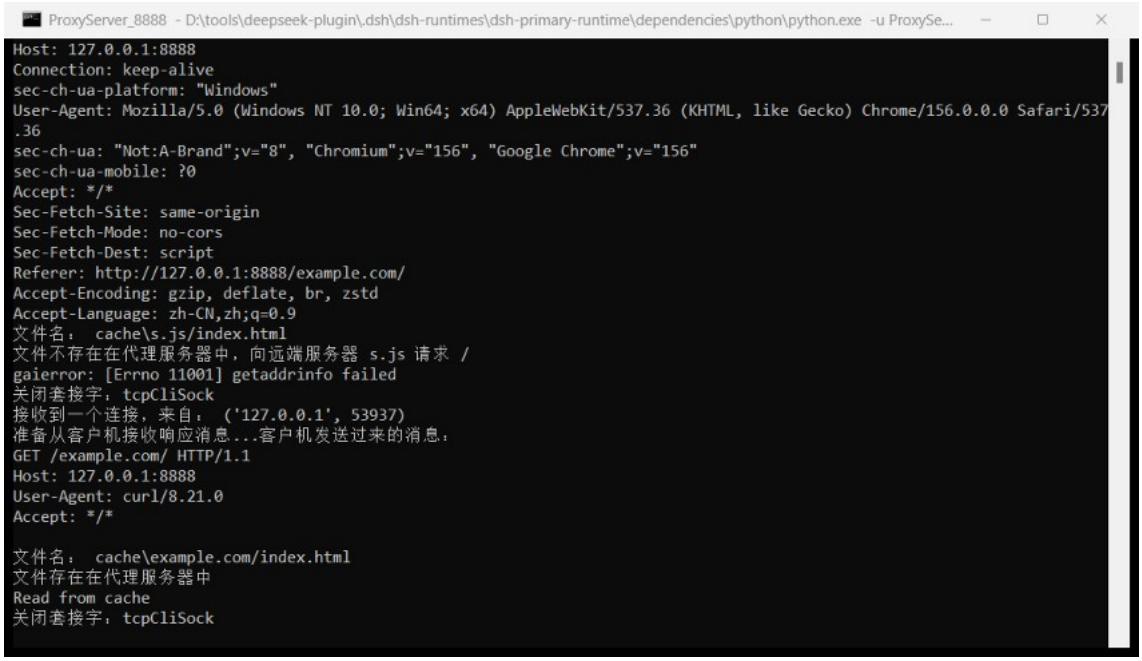


图 7 ProxyServer.py 控制台：再次访问时缓存命中，直接读本地文件

用 curl 可以看到两种情况下响应头的差别：

```
> curl -D- -o NUL http://127.0.0.1:8888/example.com/
HTTP/1.0 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 577
X-Cache: MISS

> curl -D- -o NUL http://127.0.0.1:8888/example.com/
HTTP/1.0 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 577
X-Cache: HIT
```

访问一个不存在的主机（www.baidu.c）时，域名解析失败，代理返回 404（图 8）：

```
> curl -D- -o NUL http://127.0.0.1:8888/www.baidu.c/
HTTP/1.0 404 Not Found
Content-Type: text/html; charset=utf-8
Content-Length: 106
```



图 8 通过代理访问不存在的主机时返回 404 页面

四、实验问题解答

表 2 Socket 编程实验的问题与解答

序号	问题	解答与依据
1	bind 的第一个参数为什么留空字符串？	空串等价于 0.0.0.0，表示监听本机所有网卡地址。若绑定 127.0.0.1，则只有本机能连，实验要求"从另一台主机访问"就必须绑 0.0.0.0 或具体网卡 IP。
2	为什么服务器要设置 SO_REUSEADDR？	服务器关闭后，已建立的连接会进入 TIME_WAIT 状态并占用端口约 2 分钟，此时立即重启会报 Address already in use。SO_REUSEADDR 允许在 TIME_WAIT 期间重新绑定同一端口，避免每次调试都要等两分钟。
3	recv(4096) 只读一次，会不会读不全请求？	GET 请求只有首部、没有请求体，通常 300~800 字节，远小于 4096，一次 recv 就能读完。但严格来说 TCP 是字节流，一次 recv 不保证拿到完整首部，代理服务器因为要处理各种浏览器，所以改成了循环读到 \r\n\r\n 为止。
4	响应为什么必须带 Content-Length？	它告诉客户端正文的字节数，客户端据此判断响应何时结束。缺少该首部且未用分块编码时，客户端只能等连接关闭，无法边收边渲染，页面显示会明显变慢。
5	为什么文件要用 "rb" 方式打开？	HTTP 传输的是字节流，图片、压缩包等二进制内容不能按文本解码。文本模式还会在 Windows 上把 \n 转成 \r\n，导致 Content-Length 与实际发送字节数不一致，客户端会一直等待剩余字节。
6	访问页面时服务器为什么会收到 favicon.ico 请求？	浏览器渲染完页面后会主动请求站点图标用于标签页显示，本服务器没有该文件，因此日志里出现一次 404。它是浏览器的附加行为，与页面内容无关。
7	代理为什么要把主机名拼进缓存路径？	不同站点的路径可能相同（都叫 /index.html），只用路径做文件名会相互覆盖。按 host/path 分目录既避免了冲突，也便于人工查看缓存内容。
8	回源为什么用 HTTP/1.0 + Connection: close？	HTTP/1.0 默认短连接，服务器发完即关闭，"recv 返回空"就等于响应结束，无需解析 Content-Length。若用 HTTP/1.1 长连接，读循环只能靠超时退出，每个请求都会多等数秒。
9	代理为什么必须多线程？	浏览器加载一个页面会并发请求多个对象，单线程串行处理会让每个对象排队等待，页面加载时间成倍增加；而且一个客户端不发数据就会阻塞整个服务。
10	X-Cache 头有什么作用？	它是本实验自定义的诊断首部，HIT 表示由本地缓存提供、MISS 表示回源获取。真实代理常用 Age、Via、Cache-Control 等标准首部表达同样信息。
11	请求不存在的主机时代理为什么返回 404 而不是 500？	域名解析失败（gaierror）说明"目标资源不存在"，语义上更接近 404；而 500 用于代理自身代码出错。真实的转发失败通常用 502 Bad Gateway 或 504 Gateway Timeout，本实验简化为 404/500 两种。
12	为什么用 sendall 而不是 send？	send 只保证把数据放进发送缓冲区，返回实际发送的字节数，可能小于请求发送的长度；sendall 会循环调用直到全部发完。发送响应正文这种大块数据时必须用 sendall，否则会截断。

五、实验总结

5.1 遇到的问题与解决办法

表 3 Socket 编程实验中遇到的问题与处理

遇到的问题	原因分析	解决办法
原始框架用 Python 2 语法，直接运行报 SyntaxError	print 是语句而非函数，send() 不接受 str	改写为 Python 3：print() 函数、发送前统一 encode()、接收后 decode()
端口 6789/8888 已被占用，bind 失败但程序不报错就退出	上一次调试残留的 python 进程没结束	用 netstat -ano 找到 PID 后 Stop-Process 强制结束，并给服务器加上 SO_REUSEADDR
浏览器访问时页面空白，控制台却显示已发送 200	响应头缺少 Content-Length，浏览器一直在等正文结束	在首部中补上 Content-Length 与 Connection: close
发送图片时程序抛 UnicodeDecodeError	以文本模式打开文件并按 str 发送	改为 'rb' 读取字节、sendall 发送字节
代理首次访问要等很久（约 8 秒）才显示页面	回源用了长连接，recv 循环只能靠超时退出	回源请求改为 HTTP/1.0 + Connection: close
代理返回的图片显示不出来	所有响应都硬编码 Content-Type: text/html	按扩展名判断类型（png/jpg/gif/css/js/html），写入对应的 Content-Type
访问 www.baidu.c 时代理打印 gaierror 并断开连接	该域名不存在，getaddrinfo 解析失败	捕获 gaierror 单独处理，返回 404 页面而不是让连接直接断掉
两次访问都显示 X-Cache: MISS	浏览器请求带了查询串或被当作不同路径，缓存文件名不一致	把 ? 统一替换为 _ 后再映射文件名，并用 curl 对照验证 HIT/MISS

5.2 收获与体会

第一个收获是把"HTTP 就是一个文本协议"这句话真正落实了。平时用浏览器时，网页是怎么来的完全是个黑盒；自己用 socket 拼出 "GET /HelloWorld.html HTTP/1.1\r\nHost: ...\r\n\r\n" 这几十个字符发出去，再收到一段以 "HTTP/1.1 200 OK" 开头的字节流，才发现所谓 Web 服务器不过是"读文件、拼字符串、发字节"，也理解了为什么少一个 Content-Length 页面就白屏——浏览器不知道正文有多长，只能一直等。

第二个收获是对"阻塞"有了切身的体会。作业一的单线程服务器在我的测试里完全够用，可一旦换成浏览器访问，问题立刻暴露：浏览器会同时请求页面、图片和 favicon，单线程只能一个一个处理，第二个请求必须等第一个彻底结束。改成多线程之后，日志里的"接收到一个连接"开始交错出现，响应也不再排队。这让我理解了为什么网络服务器几乎都要做并发——每个请求都可能长时间占着连接不发数据，一个阻塞的 read 就能拖垮整个服务。

第三个收获来自缓存这一层。代理加上缓存之后就成了客户端与服务器之间的一道记忆：第一次访问要回源、要等，第二次直接从磁盘读，几乎无感。但页面更新了怎么办？实验里我没有实现验证策略，这正好对应 RFC 2068 里 If-Modified-Since、ETag 的那套机制——它们是缓存能正确工作的前提，而不是可有可无的装饰。